

ML_F16_VVI Community Edition Guide

F-16C Vertical Velocity Indicator for DCS World

This guide follows the ML_F16 Community Edition document pattern used for the AOA, Speedbrake and Fuel Flow community releases. It is the approved Community v1.0.0 documentation set for the VVI build and preserves the release-control lessons learned during TFT_eSPI white-screen recovery and the R10 visual cleanup.

1. Purpose of the Community Edition

The Community Edition is a simplified, working version of the F-16C Vertical Velocity Indicator intended for cockpit builders who want to drive a 240 x 320 SPI TFT display from DCS-BIOS using an ESP32-D / ESP-WROOM-32 development board.

It provides a moving vertical velocity tape, standby/OFF behaviour, stale-data handling, rear-button diagnostics, a standard ML_F16 boot splash, and a local bench animation mode. The display is not a flight-certified instrument. It is a hobby simulator component for DCS World cockpit builds.

The R10 baseline is the approved Community v1.0.0 release baseline. It retains the proven R7 no-flicker tape renderer and adds the R8/R9/R10 visual corrections: enlarged VVI legend, corrected stacked 1000 / FPM label, full-width yellow datum line removed, yellow side datum arrows retained, negative/descent colour behaviour corrected, boot progress flicker reduced, and diagnostics refresh cleaned up.

VVI tape = moving vertical velocity scale

OFF flag = no valid DCS data yet

STALE state = DCS timeout; last valid indication retained

DIAG button = local rear-button maintenance page only

2. Aircraft and Simulation Reference

The F-16 real-aircraft reference used for cockpit context is the HAF Series Aircraft F-16C/D Blocks 50 and 52+ flight manual. It is used as a layout and systems-context reference only; this public hobby document does not reproduce operational procedures.

For the DCS implementation, the practical source of truth is the DCS-BIOS F-16C output F_16C_50_VVI. The Community sketch treats this output as the driver for the moving VVI tape. The value is decoded using captured lookup-table interpolation rather than simple linear map() scaling.

3. What This Version Includes

4. What Has Been Deliberately Removed or Kept Out of the Public Release Boundary

The Community Edition release excludes the more advanced ML_F16 project and product features. The public source follows the AOA R2-style community boundary: it includes only what is needed to build, run, validate and learn from the VVI module.

Release rule: if a feature is documented as not included, it is not present in the final public source tree. The approved Community v1.0.0 source has been stripped of internal/pro compile gates and support hooks.

5. Hardware Required

The current sketch uses this TFT object:

```
TFT_eSPI tft = TFT_eSPI();
```

Display pin allocation is configured in the active TFT_eSPI User_Setup.h or selected setup file, not inside the sketch body. This is a critical release-control point for the VVI module.

6. Basic Wiring

Figure 1 - ESP32-D to ILI9341 TFT and rear diagnostics button wiring.

Wire the display and rear diagnostic button using the same pin map configured in the active TFT_eSPI setup file. The following table is the locked VVI wiring authority for the approved Community v1.0.0 release baseline.

TFT module labels vary. MOSI may be marked SDA, SDI or DIN. DC may be marked RS or A0. Check the function, not only the printed label. The rear diagnostic button uses the ESP32 internal pull-up; no external resistor is required.

7. Required Arduino Libraries

```
#include <Arduino.h>
#include <TFT_eSPI.h>
#include <SPI.h>
#define DCSBIOS_DEFAULT_SERIAL
#include "DcsBios.h"
```

A sketch can compile successfully and still show a white TFT if TFT_eSPI is compiling against the wrong User_Setup.h or selected setup file.

8. TFT_eSPI Setup Requirement

The VVI release package must include the known-good TFT_eSPI setup file. This is now mandatory because the working VVI module previously white-screened after TFT_eSPI setup/configuration drift, while the VVI .ino itself was not at fault.

```
#define ILI9341_DRIVER
#define TFT_WIDTH 240
#define TFT_HEIGHT 320
#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS 15
#define TFT_DC 2
#define TFT_RST 4
#define SPI_FREQUENCY 27000000
#define SPI_READ_FREQUENCY 20000000
#define SPI_TOUCH_FREQUENCY 2500000
```

Recommended validation step: run TFT_eSPI -> Test and diagnostics -> Read_User_Setup before loading the VVI sketch. It must report the same driver, pins and SPI frequency shown above.

9. DCS-BIOS Requirement

The sketch expects the F-16C DCS-BIOS definitions that expose the vertical velocity output.

```
void onVviChange(unsigned int newValue) {
  updateDcsChannel(chVvi, (uint16_t)newValue);
  vviTargetKfpm = decodeVviFromRaw(chVvi.raw);
}
```

```

}
DcsBios::IntegerBuffer vviBuffer(
F_16C_50_VVI,
onVviChange
);

```

10. DCS-BIOS Serial Mode

Figure 2 - DCS World to ESP32-D VVI data flow.

The Community Edition uses DCS-BIOS default serial when live DCS mode is enabled. The ESP32-D is connected to the host PC over USB serial.

Normal use: upload the sketch to the ESP32-D, connect the ESP32-D to the DCS-BIOS host PC by USB, configure DCS-BIOS Bridge to use the ESP32 COM port, start DCS World and load the F-16C. Once F_16C_50_VVI is received, the display moves from OFF/standby through sync into live operation.

11. Main Configuration Switches

The Community sketch should contain only community-safe compile-time switches. The approved Community v1.0.0 build defaults to live DCS-BIOS mode; set bench-test switches only for local testing.

```

#define USE_DCS_BIOS 1
#define USE_TEST_VALUES 0
#define SHOW_STARTUP_SPLASH 1
#define ENABLE_DIAGNOSTICS 1
#define ENABLE_REAR_DIAG_BTN 1
#define ENABLE_TOUCH_DIAGS 0
#define SHOW_RAW_DEBUG 0

```

11.1 Normal Cockpit Use

```

#define USE_DCS_BIOS 1
#define USE_TEST_VALUES 0

```

11.2 Bench Test Without DCS

```

#define USE_DCS_BIOS 0
#define USE_TEST_VALUES 1

```

11.3 Rear Diagnostics

```

#define ENABLE_DIAGNOSTICS 1
#define ENABLE_REAR_DIAG_BTN 1
#define PIN_REAR_DIAG_BTN 27

```

12. Display Behaviour

Figure 3 - VVI tape concept and R10 static-label layout.

12.1 Startup Splash

On power-up, the display shows the standard ML_F16 startup splash, build identity, hardware target and a left-to-right progress bar. The R10 branch preserves the non-flickering incremental boot-progress behaviour.

ML_F16_VVI

1C R10 FIX

ESP32 / TFT

ML_F16 AVIONICS BOOT

12.2 Standby / OFF Mode

Before valid DCS-BIOS VVI data is received, the indicator displays an OFF condition inside the tape aperture.

12.3 Live Mode

In live mode, the vertical tape position is driven by decoded F_16C_50_VVI data. The display is a split-field tape: positive/climb indication uses a light field with dark markings; negative/descent indication uses a dark field with light markings.

12.4 Stale Mode

If DCS-BIOS data stops updating for longer than the configured timeout, the gauge enters stale mode. The last valid indication is retained with a visible stale overlay.

```
#define DCS_TIMEOUT_MS 3000
```

12.5 Community v1.0.0 Visual Baseline

13. System State Model

Figure 4 - Simplified Community VVI state model.

This keeps the Community Edition aligned with the broader ML_F16 lifecycle model while avoiding protected maintenance and productisation features.

14. VVI Decoder and Calibration

The decoder uses captured DCS raw values and interpolates between known points. This is safer than simple linear scaling because the cockpit visual behaviour is better represented by measured raw-to-indication captures.

```
float decodeVviFromRaw(uint16_t raw) {  
    return interpolateByRaw(raw, VVI_TABLE, VVI_TABLE_COUNT);  
}
```

Do not replace the lookup table with map() unless new live DCS captures prove a linear conversion is valid. For the current VVI baseline, the table is the calibration authority.

15. Tape Rendering

The renderer separates data decoding from drawing. The raw DCS value is decoded into thousands of feet per minute, then the renderer positions the moving tape inside a fixed aperture.

The physical front mask or bezel should provide the final cockpit aperture. The public sketch should not depend on protected CAD or software-only cosmetics.

16. Diagnostics Page

The Community Edition includes a simple rear-button diagnostics page. The R10 branch reduces flicker by drawing the static diagnostic labels once and updating only cached value fields at a slower cadence.

Enter/exit diagnostics using the rear momentary button wired between GPIO27 and GND. The input is active LOW and uses INPUT_PULLUP.

17. Local Test Mode

Local test mode animates the TFT and VVI tape without DCS World. Use it before live DCS testing.

```
#define USE_DCS_BIOS 0
```

```
#define USE_TEST_VALUES 1
```

Use local test mode to check TFT orientation, tape direction, split-field colours, boot screen, rear button behaviour, diagnostics page, and physical aperture alignment. Return to normal DCS mode afterwards.

```
#define USE_DCS_BIOS 1
```

```
#define USE_TEST_VALUES 0
```

18. Upload Procedure

1. Open the VVI Community Edition .ino file in the Arduino IDE.
2. Select an ESP32 Dev Module / ESP-WROOM-32 compatible board profile.
3. Install TFT_eSPI, SPI and the DCS-BIOS library/fork used by your setup.
4. Install or select User_Setup_ML_F16_VVI_ESP32_ILI9341.h in TFT_eSPI.
5. Run Read_User_Setup and confirm the active driver and pins match this guide.
6. For bench test, set USE_DCS_BIOS to 0 and USE_TEST_VALUES to 1.
7. Compile and upload to the ESP32-D.
8. Confirm the TFT shows the boot splash and moving local test tape.
9. For cockpit use, set USE_DCS_BIOS to 1 and USE_TEST_VALUES to 0.
10. Upload again, start DCS-BIOS Bridge, select the ESP32 COM port, start DCS World and load the F-16C.
11. Confirm the display moves from OFF/standby to live VVI indication.

19. Recommended Arduino IDE Settings

If upload fails, close DCS-BIOS Bridge, close Arduino Serial Monitor, confirm the COM port and use a known-good data cable.

20. DCS-BIOS / Bridge Notes

DCS World -> DCS-BIOS F-16C export -> DCS-BIOS Bridge -> ESP32-D USB serial -> TFT VVI

If the display stays in OFF/standby, the ESP32 and TFT are running but no valid VVI data is being received. Check the DCS-BIOS installation, DCS-BIOS Bridge COM port, F-16C aircraft selection, and the F_16C_50_VVI macro in the installed DCS-BIOS fork.

21. Troubleshooting

21.1 TFT White / Blank

A white TFT usually means the backlight is powered but the display controller is not receiving valid initialisation commands. This was the confirmed VVI recovery lesson: the sketch was not at fault; the active TFT_eSPI configuration had drifted.

Check TFT VCC and GND, SCLK, MOSI, CS, DC, RST and BL wiring. Confirm TFT_eSPI User_Setup.h matches the ILI9341 driver and the pin map in this guide. Run Read_User_Setup and a simple TFT_eSPI or Adafruit ILI9341 tester before testing DCS-BIOS.

21.2 Upload Fails

Check the correct COM port, use a data-capable USB cable, close DCS-BIOS Bridge and Arduino Serial Monitor during upload, and try a lower upload speed if required.

21.3 Splash Shows But Never Goes Live

Check DCS-BIOS is installed and exporting, DCS-BIOS Bridge is running, the correct ESP32 COM port is selected, the F-16C is loaded in DCS, and F_16C_50_VVI exists in the installed DCS-BIOS fork.

21.4 Rear Diagnostic Button Does Not Work

Confirm one side of the button is wired to GPIO27, the other side is wired to GND, GPIO27 is not reused by the TFT pin map, and ENABLE_REAR_DIAG_BTN is set to 1. The button is active LOW and uses the ESP32 internal pull-up.

21.5 Tape Colours Look Wrong

If red/blue or colour order appears wrong after the display works, test TFT_RGB_ORDER or TFT_BGR in the TFT_eSPI setup file. Do not change calibration or tape logic to correct a display colour-order issue.

21.6 Negative/Descent Field Inverts Incorrectly

Use the Community v1.0.0 baseline or later. R9/R10 corrected the negative/descent split-field colour behaviour after the R8 visual cleanup introduced a regression.

22. Code Structure Overview

23. Safe Areas for Community Modification

24. Areas Not Included in the Community Edition

The public release should not include PanelReporter integration, PFLD integration, fleet reporting, future networking hooks, commercial/pro build-control framework, protected support tooling, touch diagnostics, extended raw capture logging, protected mechanical CAD package, or pro display variants. These omissions are intentional. The Community Edition is a practical public learning build, not the full internal/professional branch.

25. Community Release Notes

Release text: This is the approved Community v1.0.0 F-16C Vertical Velocity Indicator sketch for DCS World. It is designed for an ESP32-D / ESP-WROOM-32 and a 2.4 inch 240 x 320 SPI TFT PCB ILI9341 using TFT_eSPI and DCS-BIOS. The display shows a cockpit-style moving VVI tape based on the F_16C_50_VVI DCS-BIOS output.

This Community v1.0.0 release includes the standard ML_F16 startup splash, Community build identity, version and edition fields, standby/OFF state, live DCS-BIOS input by default, stale timeout handling, lookup-table calibration, rear diagnostic button, local bench test mode, no-flicker sprite tape rendering, corrected 1000 / FPM label, enlarged VVI legend and the release-controlled TFT_eSPI setup file.

Advanced commercial/prototype features are not present in the approved Community v1.0.0 source tree.

26. Disclaimer

This project is a hobby cockpit-simulator component for use with DCS World. It is not affiliated with, endorsed by, or approved by Eagle Dynamics, Lockheed Martin, the United States Air Force, the Hellenic Air Force, or any aircraft manufacturer/operator. It is intended for simulation and educational use only.

27. Licence Position

28. Quick Start Summary

Build the circuit: ESP32-D plus 2.4 inch ILI9341 SPI TFT and rear diagnostic button.

Install TFT_eSPI and DCS-BIOS.

Install/select User_Setup_ML_F16_VVI_ESP32_ILI9341.h in TFT_eSPI.

Run Read_User_Setup and confirm ILI9341 plus GPIO19/23/18/15/2/4.

Open the Community Edition sketch.

For bench test, set USE_DCS_BIOS to 0 and USE_TEST_VALUES to 1.

For live cockpit use, set USE_DCS_BIOS to 1 and USE_TEST_VALUES to 0.

Upload to the ESP32-D.

Connect DCS-BIOS Bridge to the ESP32 COM port.

Start DCS World and load the F-16C.

Confirm the display moves from OFF/standby to live VVI indication.

29. Final Notes

The VVI module is now an approved Community Edition release because the core behaviour is visible, useful and educational: a builder can see DCS-BIOS raw data become a moving cockpit-style flight instrument. The release also captures an important practical lesson for the whole ML_F16 ecosystem: any TFT_eSPI module is not reproducible unless the exact active setup file is released with the sketch.

Recommended release package: INO file, this guide, README, RELEASE_NOTES, User_Setup_ML_F16_VVI_ESP32_ILI9341.h, TFT tester sketch, wiring diagram, display photos and a short demo video showing climb/descent movement, stale behaviour and the diagnostics screen.

30. Appendices

Appendix A - Community README Content Incorporated

ML_F16_VVI_COMMUNITY

Community Edition Vertical Velocity Indicator for the DCS F-16C.

Hardware:

- ESP32-D / ESP-WROOM-32 compatible board
- 2.4 inch 240 x 320 SPI TFT PCB ILI9341 configured through TFT_eSPI
- Rear diagnostic button on GPIO27 to GND
- USB serial connection to DCS-BIOS

DCS-BIOS output:

- Aircraft: F-16C_50
- Output: VVI
- Macro: F_16C_50_VVI

Community release boundary:

- Included: DCS-BIOS live VVI input, lookup-table calibration, moving tape, OFF flag, stale timeout, rear diagnostics, bench test mode and TFT_eSPI setup file.
- Not included: internal/pro edition switching, PanelReporter, PFLD, commercial support logic, future networking hooks, touch diagnostics or protected CAD package.

Appendix B - TFT_eSPI User Setup

```
// =====
```

```
// ML_F16 TFT_eSPI USER SETUP
```

```
// Module: ML_F16_VVI
```

```
// Target: ESP32-D / ESP-WROOM-32 + 2.4/2.5 inch 240x320 SPI TFT
```

```

// Driver: ILI9341
//
// Purpose:
// Clean replacement for TFT_eSPI/User_Setup.h after library update/reset.
// This file deliberately removes the default ESP8266 NodeMCU PIN_Dx block
// and leaves only one active ESP32 SPI pin definition set.
//
// Install:
// Replace the active TFT_eSPI/User_Setup.h with this file, or place this
// content in a dedicated User_Setups file and include it from
// User_Setup_Select.h.
//
// Notes:
// - VVI .ino defines only the rear diagnostic button on GPIO27.
// - TFT pins are defined here, not in the VVI sketch.
// - If the screen is white, verify CS/DC/RST against physical wiring first.
// =====
// =====

#ifndef USER_SETUP_INFO
#define USER_SETUP_INFO "ML_F16_VVI_ESP32_ILI9341_240x320"
#endif

// -----
// Display driver
// -----

#define ILI9341_DRIVER
// Portrait geometry used by ML_F16_VVI
#define TFT_WIDTH 240
#define TFT_HEIGHT 320
// Optional colour-order correction.
// Uncomment ONE only if red/blue are swapped after the display is working.
// #define TFT_RGB_ORDER TFT_RGB
// #define TFT_RGB_ORDER TFT_BGR
// Optional inversion correction.
// Uncomment ONE only if black/white or colours are inverted after the display works.
// #define TFT_INVERSION_ON
// #define TFT_INVERSION_OFF
// -----
// ESP32 SPI pin layout

```



```

// -----
//
// Hardware SPI on ESP32 VSPI-style pins.
// These must match the physical wiring.
//
// TFT labels vary by board:
// MOSI may be labelled SDA, SDI, DIN, or MOSI
// SCLK may be labelled SCK, CLK, or SCL
// DC may be labelled RS, A0, or D/C
// RST may be labelled RESET or RES
//
#define TFT_MISO 19 // TFT SDO/MISO; optional, may be left disconnected
#define TFT_MOSI 23 // TFT SDA/SDI/DIN/MOSI
#define TFT_SCLK 18 // TFT SCK/CLK
#define TFT_CS 15 // TFT CS
#define TFT_DC 2 // TFT DC/RS/A0
#define TFT_RST 4 // TFT RST/RESET/RES
// If the display RESET pin is tied to ESP32 EN/RST or to 3.3V, use this instead:
// #define TFT_RST -1
// -----
// Backlight
// -----
//
// Leave TFT_BL undefined if the display LED/BL pin is wired directly to 3.3V.
// Define TFT_BL only if the ESP32 controls the backlight via a GPIO.
//
// #define TFT_BL 32
// #define TFT_BACKLIGHT_ON HIGH
// -----
// Fonts
// -----
#define LOAD_GLCD
#define LOAD_FONT2
#define LOAD_FONT4
#define LOAD_FONT6
#define LOAD_FONT7
#define LOAD_FONT8
#define LOAD_GFXFF

```

```

#define SMOOTH_FONT
// -----
// SPI speed
// -----
//
// 27 MHz is conservative and stable for many ILI9341 modules.
// 40 MHz can be tested later once the display is confirmed stable.
//
#define SPI_FREQUENCY 27000000
#define SPI_READ_FREQUENCY 20000000
#define SPI_TOUCH_FREQUENCY 2500000
// -----
// Optional SPI port selection
// -----
//
// VSPI is the default on ESP32.
// Uncomment only if the hardware has deliberately been wired for HSPI.
//
// #define USE_HSPI_PORT
// -----
// Touch
// -----
//
// ML_F16_VVI uses rear diagnostics button only. Touch diagnostics are disabled
// in the VVI sketch, so TOUCH_CS is intentionally not defined here.
//
// #define TOUCH_CS 21

```

Appendix C - Community v1.0.0 INO Code

The separate .ino file remains the source of truth for compiling and upload. The code below is the approved Community v1.0.0 source used for this document. It defaults to live DCS-BIOS mode and excludes internal/pro compile gates, PanelReporter, PFLD, NVG proxy and future networking hooks.

```

/*****
* ML_F16_VVI_COMMUNITY
* Community Release v1.0.0
*
* F-16C Vertical Velocity Indicator
* ESP32-D / ESP-WROOM-32 + 2.4/2.5" 240x320 SPI TFT using TFT_eSPI
*

```

- * Release artefacts:
 - * - README.md
 - * - RELEASE_NOTES.md
 - * - LICENSE
 - * - User_Setup_ML_F16_VVI_ESP32_ILI9341.h
 - *
- * Community release scope:
 - * - Standard ML_F16 boot splash
 - * - Build identity, edition and version
 - * - Live DCS-BIOS VVI feed
 - * - DCS stale/no-data handling
 - * - Moving vertical tape behind fixed datum markers
 - * - OFF flag for no-data / standby
 - * - Rear diagnostic button only
 - * - Basic local diagnostics page
 - * - Window-sized sprite to reduce live-mode flicker
 - *
- * DCS-BIOS channel:
 - * F_16C_50_VVI
 - *
 - * If Arduino reports that F_16C_50_VVI is not declared, the DCS-BIOS
 - * F-16C export uses a different symbol name. Change only DCS_VVI_CHANNEL
 - * after checking the generated control reference.

```

*****/

#include <Arduino.h>
#include <TFT_eSPI.h>
#include <SPI.h>

// -----
// DCS-BIOS
// -----

#define DCSBIOS_DEFAULT_SERIAL
#include "DcsBios.h"
// -----

// BUILD INFO
// -----

#define BUILD_NAME "ML_F16_VVI"
#define BUILD_PHASE "COMMUNITY V1.0"
#define BUILD_DATE "2026-06-29"

```

```

#define BUILD_EDITION "COMMUNITY"
#define BUILD_VERSION "1.0.0"
#define PANEL_ID "F16_VVI"
#define HARDWARE_TARGET "ESP32-D / 2.4-2.5 TFT_eSPI"
#define BUILD_PHASE_SHORT "COMMUNITY V1.0"
#define HARDWARE_TARGET_SHORT "ESP32 / TFT"
// -----
// FRAMEWORK SWITCHES
// -----
// Community release defaults to live DCS-BIOS. Set USE_TEST_VALUES to 1 only for
// bench demonstration without DCS.
#define USE_DCS_BIOS 1
#define USE_TEST_VALUES 0
#define ENABLE_DIAGNOSTICS 1
#define ENABLE_REAR_DIAG_BTN 1
#define ENABLE_TOUCH_DIAGS 0
#define SHOW_STARTUP_SPLASH 1
#define SHOW_RAW_DEBUG 0
// DCS-BIOS channel under test. Change here only if the control-reference symbol
// differs.
#ifndef DCS_VVI_CHANNEL
#define DCS_VVI_CHANNEL F_16C_50_VVI
#endif
// -----
// HARDWARE CONFIG
// -----
#define SCREEN_W 240
#define SCREEN_H 320
// Rear diagnostic button.
// Wire button between GPIO27 and GND.
// Uses INPUT_PULLUP.
#define PIN_REAR_DIAG_BTN 27
// -----
// TIMING CONFIG
// -----
#define STANDARD_BOOT_TIME_MS 12000
#define STARTUP_SPLASH_MS STANDARD_BOOT_TIME_MS
#define DCS_TIMEOUT_MS 3000
#define SYNC_SWEEP_MS 1600

```

```

#define FRAME_MS 33
#define VVI_RENDER_INTERVAL_MS 80
#define VVI_REDRAW_DELTA_KFPM 0.05f
#define VVI_FILTER_ALPHA 0.14f
#define DIAG_DEBOUNCE_MS 300
#define DIAG_REFRESH_MS 1000
// -----
// DISPLAY GEOMETRY
// -----
// The screen is treated as the moving instrument mechanism behind a
// physical etched/printed VVI cover. Keep critical elements away from edges.
#define WINDOW_X 82
#define WINDOW_Y 34
#define WINDOW_W 92
#define WINDOW_H 252
#define DATUM_Y 160
#define TAPE_SCALE_X 4
#define TAPE_SCALE_W 84
#define TAPE_SCALE_R (TAPE_SCALE_X + TAPE_SCALE_W)
#define TAPE_RIGHT_BAR_X (TAPE_SCALE_R - 11)
#define TAPE_RIGHT_BAR_W 8
#define LEFT_LABEL_X 42
// VVI is represented in thousands of feet per minute.
// +1.0 = +1000 fpm, -1.0 = -1000 fpm.
#define VVI_MIN_KFPM -6.0f
#define VVI_MAX_KFPM 6.0f
// 48 px per 1000 fpm gives good visual movement while keeping +/-1 and +/-0.5
// readable in the central field, similar to the reference shots.
#define PIXELS_PER_KFPM 80.0f
// -----
// COLOURS
// -----
#define COL_BLACK TFT_BLACK
#define COL_WHITE TFT_WHITE
#define COL_GREEN TFT_GREEN
#define COL_BLUE TFT_BLUE
#define COL_RED TFT_RED
#define COL_YELLOW TFT_YELLOW

```

```

#define COL_GREY 0x8410
#define COL_DIM 0x4208
#define COL_AMBER 0xFD20
#define COL_OFF_ORANGE 0xFBEO
#define COL_OFF_RED 0xD2A0
#define COL_STALE_DIM 0x7800
// -----
// SYSTEM STATES
// -----
enum SystemState {
SYS_OFF,
SYS_BOOT,
SYS_STANDBY,
SYS_SYNC,
SYS_LIVE,
SYS_STALE,
SYS_DIAG
};
SystemState currentState = SYS_BOOT;
SystemState lastRenderedState = SYS_OFF;
// -----
// DCS CHANNEL STRUCTURE
// -----
struct DcsChannel {
const char* name;
uint16_t raw;
float decoded;
unsigned long lastUpdateMs;
uint32_t updateCount;
bool live;
};
DcsChannel chVvi = {
"VVI",
0,
0.0f,
0,
0,
false

```

```

};
// -----
// CALIBRATION TABLE
// -----

struct CalPoint {
float value;
uint16_t raw;
};
/*
PHASE 1C R4 calibrated VVI table.
Captured live in DCS F-16C using the VVI cockpit indication.
Units are thousands of feet per minute:
+1.0 = +1000 fpm
-1.0 = -1000 fpm
Note:
+0.5 was not captured in this pass. The interpolation function will
calculate it between the validated 0.0 and +1.0 points. Capture +0.5
later if we want a fully symmetrical 500 fpm table.
*/
CalPoint vviCalTable[] = {
{ -6.0f, 7601 },
{ -5.5f, 9726 },
{ -5.0f, 11855 },
{ -4.5f, 13966 },
{ -4.0f, 15652 },
{ -3.5f, 18332 },
{ -3.0f, 19998 },
{ -2.5f, 22644 },
{ -2.0f, 24462 },
{ -1.5f, 26940 },
{ -1.0f, 28545 },
{ -0.5f, 30556 },
{ 0.0f, 32574 },
{ 1.0f, 38196 },
{ 1.5f, 41223 },
{ 2.0f, 43318 },
{ 2.5f, 46382 },
{ 3.0f, 49327 },

```

```

{ 3.5f, 52168 },
{ 4.0f, 54581 },
{ 4.5f, 57452 },
{ 5.0f, 60142 },
{ 5.5f, 62891 },
{ 6.0f, 65492 }
};

const int VVI_CAL_COUNT = sizeof(vviCalTable) / sizeof(vviCalTable[0]);

// -----
// DISPLAY OBJECTS
// -----
TFT_eSPI tft = TFT_eSPI();
TFT_eSprite tape = TFT_eSprite(&tft);
// -----
// LIVE VALUES
// -----

float vviTargetKfpm = 0.0f;
float vviDisplayedKfpm = 0.0f;
float vviLastRenderedKfpm = 999.0f;
float testVviKfpm = -1.0f;
bool testDirectionUp = true;
bool dcsEverSeen = false;
bool diagnosticsMode = false;
bool splashComplete = false;
bool bootStaticDrawn = false;
int bootLastProgressW = -1;
bool tapeReady = false;
bool staticFaceDrawn = false;
bool diagnosticsDrawn = false;
bool offFlagDrawn = false;
unsigned long bootMs = 0;
unsigned long lastFrameMs = 0;
unsigned long lastTapeRenderMs = 0;
unsigned long lastDcsPacketMs = 0;
unsigned long syncStartMs = 0;
unsigned long lastDiagButtonMs = 0;
unsigned long lastDiagRefreshMs = 0;
uint32_t frameCounter = 0;

```



```

uint32_t staleCount = 0;
// -----
// BASIC HELPERS
// -----

float clampFloat(float v, float lo, float hi) {
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

float lerpFloat(float a, float b, float t) {
    return a + ((b - a) * t);
}

float interpolateTable(uint16_t raw, const CalPoint* table, int count) {
    if (count <= 0) return 0.0f;
    if (raw <= table[0].raw) return table[0].value;
    if (raw >= table[count - 1].raw) return table[count - 1].value;
    for (int i = 0; i < count - 1; i++) {
        if (raw >= table[i].raw && raw <= table[i + 1].raw) {
            float span = (float)(table[i + 1].raw - table[i].raw);
            if (span <= 0.0f) return table[i].value;
            float t = (float)(raw - table[i].raw) / span;
            return lerpFloat(table[i].value, table[i + 1].value, t);
        }
    }
    return table[0].value;
}

const char* stateName(SystemState s) {
    switch (s) {
        case SYS_OFF: return "OFF";
        case SYS_BOOT: return "BOOT";
        case SYS_STANDBY: return "STANDBY";
        case SYS_SYNC: return "SYNC";
        case SYS_LIVE: return "LIVE";
        case SYS_STALE: return "STALE";
        case SYS_DIAG: return "DIAG";
        default: return "UNKNOWN";
    }
}

```

```

uint16_t cockpitWhite() {
    if (currentState == SYS_STALE) return COL_RED;
    return COL_WHITE;
}

uint16_t cockpitDim() {
    if (currentState == SYS_STALE) return COL_STALE_DIM;
    return COL_GREY;
}

void invalidateStaticFace() {
    staticFaceDrawn = false;
    diagnosticsDrawn = false;
    offFlagDrawn = false;
    vviLastRenderedKfpm = 999.0f;
    lastTapeRenderMs = 0;
}

// -----
// DCS CHANNEL UPDATE
// -----

void updateVviChannel(uint16_t newValue) {
    chVvi.raw = newValue;
    chVvi.decoded = interpolateTable(newValue, vviCalTable, VVI_CAL_COUNT);
    chVvi.decoded = clampFloat(chVvi.decoded, VVI_MIN_KFPM, VVI_MAX_KFPM);
    chVvi.lastUpdateMs = millis();
    chVvi.updateCount++;
    chVvi.live = true;
    lastDcsPacketMs = millis();
    if (!dcsEverSeen) {
        dcsEverSeen = true;
        syncStartMs = millis();
        invalidateStaticFace();
    }
}

// -----
// DCS-BIOS CALLBACKS
// -----

void onVviChange(unsigned int newValue) {
    updateVviChannel((uint16_t)newValue);
    vviTargetKfpm = chVvi.decoded;
}

```

```

}
#ifdef USE_DCS_BIOS
DcsBios::IntegerBuffer vviBuffer(
DCS_VVI_CHANNEL,
onVviChange
);
#endif

// -----
// DISPLAY SETUP
// -----

void setupDisplay() {
tft.init();
tft.setRotation(0);
tft.fillScreen(COL_BLACK);
tape.setColorDepth(16);
tape.createSprite(WINDOW_W, WINDOW_H);
tape.setTextDatum(MC_DATUM);
tape.setTextFont(2);
tapeReady = true;
}

// -----
// STANDARD ML_F16 STARTUP SPLASH
// -----

void drawStandardBootStatic() {
tft.fillScreen(COL_BLACK);
tft.setTextDatum(MC_DATUM);
tft.setTextFont(2);
tft.setTextColor(COL_AMBER, COL_BLACK);
tft.drawString("ML_F16 VVI", SCREEN_W / 2, 54);
tft.setTextFont(1);
tft.setTextColor(COL_GREY, COL_BLACK);
tft.drawString(BUILD_PHASE_SHORT, SCREEN_W / 2, 82);
tft.drawString(HARDWARE_TARGET_SHORT, SCREEN_W / 2, 100);
tft.drawString(BUILD_DATE, SCREEN_W / 2, 118);
tft.setTextFont(2);
tft.setTextColor(COL_WHITE, COL_BLACK);
tft.drawString("INITIALISING", SCREEN_W / 2, 160);
tft.drawString("AVIONICS", SCREEN_W / 2, 182);

```

```

const int barX = 30;
const int barY = 214;
const int barW = SCREEN_W - 60;
const int barH = 14;
tft.drawRoundRect(barX, barY, barW, barH, 4, COL_GREY);
tft.setTextFont(1);
tft.setTextColor(COL_DIM, COL_BLACK);
tft.drawString(PANEL_ID, SCREEN_W / 2, 252);
tft.drawString("ML_F16 MODULE", SCREEN_W / 2, 292);
bootStaticDrawn = true;
bootLastProgressW = 0;
}

void updateStandardBootProgress(float progress) {
progress = clampFloat(progress, 0.0f, 1.0f);
if (!bootStaticDrawn) drawStandardBootStatic();
const int outerX = 30;
const int outerY = 214;
const int outerW = SCREEN_W - 60;
const int outerH = 14;
const int inset = 3;
const int barX = outerX + inset;
const int barY = outerY + inset;
const int barW = outerW - (inset * 2);
const int barH = outerH - (inset * 2);
const int fillW = (int)(barW * progress);
// R9: no boot-screen flicker.
// Do not clear and redraw the whole bar every frame. The startup progress
// only moves left-to-right, so draw only the newly exposed strip.
if (bootLastProgressW < 0 || fillW < bootLastProgressW) {
tft.fillRect(barX, barY, barW, barH, COL_BLACK);
bootLastProgressW = 0;
}
if (fillW > bootLastProgressW) {
tft.fillRect(barX + bootLastProgressW,
barY,
fillW - bootLastProgressW,
barH,
COL_AMBER);

```

```

bootLastProgressW = fillW;
}
}
void runStartupSequence() {
#if SHOW_STARTUP_SPLASH
unsigned long startMs = millis();
bootStaticDrawn = false;
bootLastProgressW = -1;
while ((millis() - startMs) < STANDARD_BOOT_TIME_MS) {
#if USE_DCS_BIOS
DcsBios::loop();
#endif
float p = (float)(millis() - startMs) / (float)STANDARD_BOOT_TIME_MS;
updateStandardBootProgress(p);
delay(80);
}
updateStandardBootProgress(1.0f);
delay(200);
#else
delay(250);
#endif
splashComplete = true;
bootStaticDrawn = false;
}
// -----
// REAR DIAGNOSTIC BUTTON
// -----
void updateRearDiagnosticButton() {
#if ENABLE_DIAGNOSTICS && ENABLE_REAR_DIAG_BTN
static bool lastPressed = false;
bool pressed = digitalRead(PIN_REAR_DIAG_BTN) == LOW;
// R10: edge-triggered toggle only.
// The previous level-triggered handling could toggle repeatedly if the
// rear button was held, causing unnecessary full-screen redraws/flicker.
if (pressed && !lastPressed && (millis() - lastDiagButtonMs > DIAG_DEBOUNCE_MS)) {
diagnosticsMode = !diagnosticsMode;
lastDiagButtonMs = millis();
tft.fillScreen(COL_BLACK);

```

```

invalidateStaticFace();
}
lastPressed = pressed;
#endif
}
// -----
// TEST VALUE GENERATOR
// -----
void updateTestValues() {
#if USE_TEST_VALUES
static unsigned long lastTestMs = 0;
if (millis() - lastTestMs > 60) {
lastTestMs = millis();
testVviKfpm += testDirectionUp ? 0.025f : -0.025f;
if (testVviKfpm >= 1.25f) {
testVviKfpm = 1.25f;
testDirectionUp = false;
}
if (testVviKfpm <= -1.25f) {
testVviKfpm = -1.25f;
testDirectionUp = true;
}
vviTargetKfpm = testVviKfpm;
if (!dcsEverSeen) {
dcsEverSeen = true;
lastDcsPacketMs = millis();
syncStartMs = millis();
invalidateStaticFace();
}
}
#endif
}
// -----
// SYSTEM STATE MACHINE
// -----
void updateSystemState() {
SystemState oldState = currentState;
if (!splashComplete) {

```

```

currentState = SYS_BOOT;
} else if (diagnosticsMode) {
currentState = SYS_DIAG;
} else {
#if USE_TEST_VALUES
if (dcsEverSeen && ((millis() - syncStartMs) < SYNC_SWEEP_MS)) {
currentState = SYS_SYNC;
} else {
currentState = SYS_LIVE;
}
#else
if (!dcsEverSeen) {
currentState = SYS_STANDBY;
} else if ((millis() - lastDcsPacketMs) > DCS_TIMEOUT_MS) {
currentState = SYS_STALE;
} else if ((millis() - syncStartMs) < SYNC_SWEEP_MS) {
currentState = SYS_SYNC;
} else {
currentState = SYS_LIVE;
}
#endif
}
if (oldState != currentState) invalidateStaticFace();
}
// -----
// STATIC OVERLAY
// -----
void drawStaticLabels() {
uint16_t white = cockpitWhite();
// R5: wider split-field VVI layout based on the real/reference unit.
// The physical front plate will mask the outer display area, so software
// only renders the instrument markings that need to appear behind the aperture.
// R8/R9: left stacked VVI legend increased by two TFT_eSPI font levels.
// Moving tape numerals and the lower unit legend are intentionally unchanged.
tft.setTextDatum(MC_DATUM);
tft.setTextFont(4);
tft.setTextColor(white, COL_BLACK);
tft.drawString("V", LEFT_LABEL_X + 4, 62);

```

```

tft.drawString("V", LEFT_LABEL_X + 4, 94);
tft.drawString("I", LEFT_LABEL_X + 4, 126);
// Left lower units legend, stacked and centre-aligned like the reference unit.
// R10: corrected from uneven FTMIN text to aligned 1000 / FPM.
const int unitX = LEFT_LABEL_X + 4;
tft.setTextDatum(MC_DATUM);
tft.setTextFont(1);
tft.setTextColor(white, COL_BLACK);
tft.drawString("1", unitX, 184);
tft.drawString("0", unitX, 196);
tft.drawString("0", unitX, 208);
tft.drawString("0", unitX, 220);
tft.drawString("F", unitX, 240);
tft.drawString("P", unitX, 252);
tft.drawString("M", unitX, 264);
// No software bezel/window outline. The physical cover/bezel provides the aperture.
}

void drawDatumOverlay() {
uint16_t arrow = COL_YELLOW;
// Fixed yellow datum arrows, aligned to the wider tape window.
tft.fillTriangle(WINDOW_X - 1, DATUM_Y,
WINDOW_X - 22, DATUM_Y - 10,
WINDOW_X - 22, DATUM_Y + 10,
arrow);
tft.fillTriangle(WINDOW_X + WINDOW_W + 1, DATUM_Y,
WINDOW_X + WINDOW_W + 22, DATUM_Y - 10,
WINDOW_X + WINDOW_W + 22, DATUM_Y + 10,
arrow);
// R8/R9: no full-width yellow horizontal line.
// Datum is represented only by the two fixed side arrows.
}

void drawStaticInstrumentFace() {
tft.fillScreen(COL_BLACK);
drawStaticLabels();
drawDatumOverlay();
staticFaceDrawn = true;
}
// -----

```



```

// MOVING VVI TAPE, WINDOW-SPRITE ONLY
// -----
int vviToTapeY(float tapeCentreKfpm, float valueKfpm) {
float delta = valueKfpm - tapeCentreKfpm;
return (WINDOW_H / 2) - (int)(delta * PIXELS_PER_KFPM);
}

void drawVviMajorMark(int y, float value) {
const bool climbSide = (value >= 0.0f);
const uint16_t fg = climbSide ? COL_BLACK : cockpitWhite();
const uint16_t bg = climbSide ? COL_WHITE : COL_BLACK;
// Major ticks are longer and sparse. Numerals are deliberately restricted:
// - 0
// - .5 immediately above/below zero
// - whole-number marks only
// This prevents duplicate-looking numerals such as 1 at both 1.0 and 1.5.
tape.drawFastHLine(TAPE_SCALE_X + 1, y, 46, fg);
const float av = fabs(value);
bool showLabel = false;
String label;
if (av < 0.01f) {
showLabel = true;
label = "0";
} else if (fabs(av - 0.5f) < 0.01f) {
showLabel = true;
label = ".5";
} else if (fabs(av - roundf(av)) < 0.01f) {
showLabel = true;
label = String((int)av);
}
if (showLabel) {
tape.setTextDatum(MC_DATUM);
tape.setTextFont(4);
tape.setTextColor(fg, bg);
tape.drawString(label, TAPE_SCALE_X + 58, y);
}
}

void drawVviMinorMark(int y, float value) {
const bool climbSide = (value >= 0.0f);

```

```

const uint16_t fg = climbSide ? COL_BLACK : cockpitWhite();
// Short minor tick. No numerals.
tape.drawFastHLine(TAPE_SCALE_X + 1, y, 28, fg);
}

void drawZeroBug(int zeroY) {
// The real VVI has a dominant centre zero block riding on the moving tape.
// Draw it in the tape sprite so it moves correctly with vertical speed.
const int boxW = 30;
const int boxH = 40;
const int boxX = TAPE_SCALE_X + 45;
const int boxY = zeroY - (boxH / 2);
if (boxY > -boxH && boxY < WINDOW_H) {
tape.fillRoundRect(boxX, boxY, boxW, boxH, 3, COL_BLACK);
tape.drawRoundRect(boxX, boxY, boxW, boxH, 3, cockpitDim());
tape.setTextDatum(MC_DATUM);
tape.setTextFont(4);
tape.setTextColor(COL_WHITE, COL_BLACK);
tape.drawString("0", boxX + boxW / 2, zeroY);
}
}

void drawMovingTapeWindow(float vviKfpm, bool stale) {
if (!tapeReady) return;
tape.fillSprite(COL_BLACK);
// R9 split-field repair:
// climb field above the moving zero line = white field / black markings
// descent field below the moving zero line = black field / white markings
// When the zero line moves above the visible aperture during descent, the
// whole aperture must remain black. Do not fill the full tape white.
int zeroY = vviToTapeY(vviKfpm, 0.0f);
zeroY = constrain(zeroY, -WINDOW_H, WINDOW_H * 2);
if (zeroY >= WINDOW_H) {
// Entire aperture is above zero/climb side.
tape.fillRect(TAPE_SCALE_X, 0, TAPE_SCALE_W, WINDOW_H, COL_WHITE);
} else if (zeroY > 0) {
// Only the visible portion above the zero line is climb/white.
tape.fillRect(TAPE_SCALE_X, 0, TAPE_SCALE_W, zeroY, COL_WHITE);
}
// Distinct right-hand white reference bar visible in the descent region,

```

```

// as seen on the reference instrument.
if (zeroY < WINDOW_H) {
int barY = max(zeroY, 0);
int barH = WINDOW_H - barY;
tape.fillRect(TAPE_RIGHT_BAR_X, barY, TAPE_RIGHT_BAR_W, barH, COL_WHITE);
}
// Window side lines, intentionally light.
uint16_t edge = stale ? cockpitDim() : cockpitWhite();
tape.drawFastVLine(TAPE_SCALE_X, 0, WINDOW_H, edge);
tape.drawFastVLine(TAPE_SCALE_R - 1, 0, WINDOW_H, edge);
// Sparse markings: minor every 0.1, major every 0.5. Numerals on majors only.
// With 80 px / 1000 fpm this keeps the calibrated movement while widening the visible
tape.
for (int i = -60; i <= 60; i++) {
float value = (float)i * 0.1f;
int y = vviToTapeY(vviKfpm, value);
if (y < -18 || y > WINDOW_H + 18) continue;
if (i % 5 == 0) {
drawVviMajorMark(y, value);
} else {
drawVviMinorMark(y, value);
}
}
drawZeroBug(zeroY);
tft.startWrite();
// R7 no-flicker path:
// The sprite is opaque and covers the complete tape aperture, so do NOT
// clear the TFT window first. Clearing the live TFT before push creates
// the visible black flash/flicker reported in R6A.
tape.pushSprite(WINDOW_X, WINDOW_Y);
tft.endWrite();
}
// -----
// OFF FLAG
// -----
void drawOffFlag() {
int flagX = WINDOW_X + 12;
int flagY = DATUM_Y - 36;

```

```

int flagW = WINDOW_W - 24;
int flagH = 72;
tft.fillRect(WINDOW_X, WINDOW_Y, WINDOW_W, WINDOW_H, COL_BLACK);
tft.fillRoundRect(flagX, flagY, flagW, flagH, 2, COL_OFF_ORANGE);
tft.drawRoundRect(flagX, flagY, flagW, flagH, 2, COL_OFF_RED);
tft.setTextDatum(MC_DATUM);
tft.setTextFont(4);
tft.setTextColor(COL_BLACK, COL_OFF_ORANGE);
tft.drawString("OFF", WINDOW_X + (WINDOW_W / 2), DATUM_Y);
}
// -----
// RENDERERS
// -----
void renderInstrumentLive(bool stale) {
if (!staticFaceDrawn) drawStaticInstrumentFace();
const unsigned long now = millis();
const bool renderDue = (now - lastTapeRenderMs) >= VVI_RENDER_INTERVAL_MS;
const bool movedEnough = fabs(vviDisplayedKfpm - vviLastRenderedKfpm) >=
VVI_REDRAW_DELTA_KFPM;
if (renderDue && movedEnough) {
drawMovingTapeWindow(vviDisplayedKfpm, stale);
// Datum arrows must be redrawn only after a tape sprite update, because
// the sprite necessarily overwrites the centre datum line. Drawing them
// every frame adds unnecessary TFT traffic and can look like flicker.
drawDatumOverlay();
vviLastRenderedKfpm = vviDisplayedKfpm;
lastTapeRenderMs = now;
}
#ifdef SHOW_RAW_DEBUG
tft.fillRect(0, 0, 100, 26, COL_BLACK);
tft.setTextDatum(TL_DATUM);
tft.setTextFont(1);
tft.setTextColor(COL_GREEN, COL_BLACK);
tft.drawString("RAW " + String(chVvi.raw), 2, 2);
tft.drawString("VVI " + String(vviDisplayedKfpm, 2), 2, 14);
#endif
if (stale) staleCount++;
}

```

```

void renderStandby() {
    if (!staticFaceDrawn) drawStaticInstrumentFace();
    if (!offFlagDrawn) {
        drawOffFlag();
        drawDatumOverlay();
        offFlagDrawn = true;
    }
}

void renderSync() {
    if (!staticFaceDrawn) drawStaticInstrumentFace();
    unsigned long elapsed = millis() - syncStartMs;
    float t = clampFloat((float)elapsed / (float)SYNC_SWEEP_MS, 0.0f, 1.0f);
    float syncValue = lerpFloat(0.0f, vviTargetKfpm, t);
    drawMovingTapeWindow(syncValue, false);
    drawDatumOverlay();
    tft.fillRect(100, 296, 42, 14, COL_BLACK);
    tft.setTextDatum(MC_DATUM);
    tft.setTextFont(1);
    tft.setTextColor(COL_GREEN, COL_BLACK);
    tft.drawString("SYNC", 120, 304);
}

void renderDiagnostics() {
    #if ENABLE_DIAGNOSTICS
    static String cacheBuild;
    static String cachePhase;
    static String cacheHw;
    static String cacheState;
    static String cacheDcs;
    static String cacheAge;
    static String cacheUpdates;
    static String cacheRaw;
    static String cacheKfpm;
    static String cacheFpm;
    static String cacheDisplay;
    static String cacheStale;
    static String cacheFrames;
    static String cacheEdition;
    static String cacheVersion;
    #endif
}

```

```
if (!diagnosticsDrawn) {
tft.fillScreen(COL_BLACK);
tft.setTextDatum(TL_DATUM);
tft.setTextFont(2);
tft.setTextColor(COL_AMBER, COL_BLACK);
tft.drawString("VVI MAINT", 8, 8);
tft.setTextFont(1);
tft.setTextColor(COL_GREY, COL_BLACK);
tft.drawString("BUILD", 8, 34);
tft.drawString("PHASE", 8, 48);
tft.drawString("HW", 8, 62);
tft.drawString("STATE", 8, 88);
tft.drawString("DCS", 8, 102);
tft.drawString("AGE", 8, 116);
tft.drawString("UPDATES", 8, 130);
tft.drawString("VVI RAW", 8, 154);
tft.drawString("KFPM", 8, 168);
tft.drawString("FPM", 8, 182);
tft.drawString("DISPLAY", 8, 196);
tft.drawString("EDITION", 8, 220);
tft.drawString("VERSION", 8, 234);
tft.drawString("STALE CT", 8, 248);
tft.drawString("FRAMES", 8, 262);
tft.setTextColor(COL_GREEN, COL_BLACK);
tft.drawString("Rear button exits", 8, 306);
// R10: force first value draw after the static diagnostic page is drawn.
cacheBuild = "\x01";
cachePhase = "\x01";
cacheHw = "\x01";
cacheState = "\x01";
cacheDcs = "\x01";
cacheAge = "\x01";
cacheUpdates = "\x01";
cacheRaw = "\x01";
cacheKfpm = "\x01";
cacheFpm = "\x01";
cacheDisplay = "\x01";
cacheStale = "\x01";
```

```

cacheFrames = "\x01";
cacheEdition = "\x01";
cacheVersion = "\x01";
diagnosticsDrawn = true;
lastDiagRefreshMs = 0;
}
if ((millis() - lastDiagRefreshMs) < DIAG_REFRESH_MS) return;
lastDiagRefreshMs = millis();
const int valueX = 82;
const int valueW = 152;
auto drawValueIfChanged = [&](int y, const String& value, String& cache) {
if (value == cache) return;
// R10: use TFT_eSPI text padding rather than full field clear.
// This reduces the small flicker visible on the diagnostics page while
// still erasing leftover characters from longer previous values.
tft.setTextDatum(TL_DATUM);
tft.setTextFont(1);
tft.setTextColor(COL_WHITE, COL_BLACK);
tft.setTextPadding(valueW);
tft.drawString(value, valueX, y);
tft.setTextPadding(0);
cache = value;
};
drawValueIfChanged(34, "VVI", cacheBuild);
drawValueIfChanged(48, BUILD_PHASE_SHORT, cachePhase);
drawValueIfChanged(62, HARDWARE_TARGET_SHORT, cacheHw);
drawValueIfChanged(88, stateName(currentState), cacheState);
drawValueIfChanged(102, dcsEverSeen ? "SEEN" : "NO DATA", cacheDcs);
String ageText;
if (lastDcsPacketMs == 0) {
ageText = "NEVER";
} else {
unsigned long ageMs = millis() - lastDcsPacketMs;
if (ageMs < 1000UL) {
ageText = "<1 s";
} else {
ageText = String(ageMs / 1000UL) + " s";
}
}

```

```

}
drawValuelfChanged(116, ageText, cacheAge);
drawValuelfChanged(130, String(chVvi.updateCount), cacheUpdates);
drawValuelfChanged(154, String(chVvi.raw), cacheRaw);
drawValuelfChanged(168, String(chVvi.decoded, 2), cacheKfpm);
drawValuelfChanged(182, String((int)(vviTargetKfpm * 1000.0f)), cacheFpm);
// Quantise display value on the diagnostic page so normal smoothing does
// not force a visible redraw every maintenance refresh.
drawValuelfChanged(196, String(vviDisplayedKfpm, 1), cacheDisplay);
drawValuelfChanged(220, BUILD_EDITION, cacheEdition);
drawValuelfChanged(234, BUILD_VERSION, cacheVersion);
drawValuelfChanged(248, String(staleCount), cacheStale);
// Frame counter changes continuously, so quantise it. This keeps a useful
// health indication without forcing a visible redraw every diagnostic pass.
drawValuelfChanged(262, String(frameCounter / 100), cacheFrames);
#endif
}

void renderCurrentState() {
switch (currentState) {
case SYS_BOOT:
break;
case SYS_STANDBY:
renderStandby();
break;
case SYS_SYNC:
renderSync();
break;
case SYS_LIVE:
renderInstrumentLive(false);
break;
case SYS_STALE:
renderInstrumentLive(true);
break;
case SYS_DIAG:
renderDiagnostics();
break;
case SYS_OFF:
default:

```



```

tft.fillScreen(COL_BLACK);
break;
}
lastRenderedState = currentState;
}
// -----
// SETUP
// -----
void setup() {
bootMs = millis();
#if ENABLE_REAR_DIAG_BTN
pinMode(PIN_REAR_DIAG_BTN, INPUT_PULLUP);
#endif
setupDisplay();
currentState = SYS_BOOT;
lastRenderedState = SYS_BOOT;
runStartupSequence();
#if USE_DCS_BIOS
DcsBios::setup();
#endif
#if USE_TEST_VALUES
dcsEverSeen = true;
lastDcsPacketMs = millis();
syncStartMs = millis();
#else
currentState = SYS_STANDBY;
#endif
invalidateStaticFace();
lastFrameMs = millis();
}
// -----
// LOOP
// -----
void loop() {
#if USE_DCS_BIOS
DcsBios::loop();
#endif
updateRearDiagnosticButton();

```

```
updateTestValues();
updateSystemState();
vviTargetKfpm = clampFloat(vviTargetKfpm, VVI_MIN_KFPM, VVI_MAX_KFPM);
// Smooth tape motion. VVI should not snap unless we later model power loss.
vviDisplayedKfpm += (vviTargetKfpm - vviDisplayedKfpm) * VVI_FILTER_ALPHA;
vviDisplayedKfpm = clampFloat(vviDisplayedKfpm, VVI_MIN_KFPM, VVI_MAX_KFPM);
if ((millis() - lastFrameMs) >= FRAME_MS) {
    lastFrameMs = millis();
    frameCounter++;
    renderCurrentState();
}
}
```